Project 18 (Real-World Example) — ResearchPilot-Planner-Executor-Research-Agent-With-Cited-Reports

Type: Planner-executor agent (RAG) · **Stack:** Python, LangChain, Chroma (RAG), OpenAI SDK, Pydantic · **Level:** Intermediate

Tagline: Ask a broad question; the agent **plans** it into sub-questions, **researches each one** against your documents, and writes a report where every claim is **cited** back to its source.

Charter (what this project is)

A single RAG call answers a single question. But "give me a briefing on X" is really *several* questions, and the answer should say *where each part came from*. **ResearchPilot** is the plan-first agent (Doc07) pointed at research: a **planner** turns the broad question into a list of focused sub-questions, an **executor** searches your documents for each one, and a **compiler** assembles the findings into a report with numbered **citations**. The plan is written down before anything runs, so you can see — and trust — how the answer was built.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3

The Story — what you're building

A broad question comes in. The **planner** (a structured LLM call) breaks it into a short list of sub-questions. The **executor** runs each sub-question through a **search** function — a RAG lookup over your document store — and collects a `Finding` (the retrieved text plus its source) for each. A deterministic **compiler** folds the findings into a `Report`, and a **formatter** prints it with numbered citations, de-duplicating sources so the same document is `[1]` throughout.

The lesson: a research agent is **plan → execute each step → compile**, and the thing that makes it *trustworthy* is citations — every claim points to the document it came from, so a reader can verify it. The executor takes a `search` function (not a hard-wired store), so the whole plan-and-compile engine is testable with a fake search, no model or embeddings.

Project Structure (the professional layout)

`main.py` takes a research question; all code in `src/`.

```
<repo folder>/ (the ResearchPilot repo — full name in the heading above)
├── data/sources/               a few .md documents to research over
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── source_store.py     build_source_store()      (Step 2)
│   │   └── search.py           make_search()             (Step 2)
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── executor.py             execute_plan()            (Step 1)
│   ├── planner.py              make_plan()               (Step 2)
│   ├── pipeline.py             run_research()            (Step 3)
│   ├── report.py               compile_report(), format_report() (Step 1)
│   └── schemas.py              SubQuestions, Finding, Report (Step 1)
├── tests/
│   ├── integration/test_live_research.py
│   └── unit/                   test_config, test_report
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds the standard setup, `src/schemas.py`, `src/executor.py` and `src/report.py` (the deterministic engine) with tests — using a fake search, no model.
- **Step 2** adds `src/planner.py` (the structured planner) and `src/rag/` (the document store + a `search` function).
- **Step 3** adds `src/pipeline.py`, `main.py`, and the integration test.

Setup (do this once, before Step 1)

```
cd ResearchPilot-Planner-Executor-Research-Agent-With-Cited-Reports
python -m venv .venv
```

```
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langchain-openai langchain-community langchain-text-splitters \
    chromadb openai pydantic python-dotenv
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. Write 3-5 short `.md` documents in `data/sources/` on a topic you can quiz (each file is a "source" that will be cited). `LICENSE` is the MIT text.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; this builds on Doc05/08, not pytest):

- **`tests/unit/`** — no model, no embeddings: `test_config`; `test_report` (the executor + compiler + formatter, with a **fake** `search(q) -> (text, source)` — the findings land in the report, and citations number and de-duplicate sources correctly).
- **`tests/integration/test_live_research.py`** — real model + RAG: a broad question is planned into sub-questions, each researched, and the report names findings with citations to the real source files.

Each step ends with a **Tests for this step** box.

Step 1 — The Report Shape, the Executor, and the Compiler

What this step does: the typed shapes, the deterministic `execute_plan` (runs a `search` per sub-question), and `compile_report` / `format_report` (with numbered, de-duplicated citations) — all proven with a fake search, no model.

Why this step matters: the engine — run each step, collect findings, cite sources — is the whole project minus the model. Because `execute_plan` takes a `search` function, you test every path (findings, citation numbering, source de-duplication) with made-up data.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write the standard setup (`config` / `exceptions` / `utils/logger` / `models/llm`). 2. Write `src/schemas.py`: `SubQuestions(sub_questions)`, `Finding(sub_question, answer, source)`, `Report(question, findings)`. 3. Write `src/executor.py`: `execute_plan(sub_questions, search) -> list[Finding]` — call `search(sq)` (returns `(text, source)`) for each sub-question. Write `src/report.py`: `compile_report(question, findings)` and `format_report(report)` — number the sources, cite each finding with its source's number, and list the de-duplicated sources at the end.

Tests for this step: `test_report` — with a fake `search`, the findings appear, two findings from the same source share one citation number, and the Sources list has no duplicates. `test_config`.

Step 2 — The Planner and the RAG Search

What this step does: the planner (a structured LLM call that breaks the question into sub-questions) and the RAG search (a document store + a `search(q) -> (text, source)` the executor calls).

Why this step matters: the planner is the "plan-first" idea (Doc07) made concrete, and the search is what grounds each finding in a real, citable document.

What's new vs. Step 1: `src/planner.py`, `src/rag/*`. **What stays the same:** the executor, compiler, schemas.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. `src/planner.py`: `make_plan(model, question) -> SubQuestions` = prompt | `model.with_structured_output(SubQuestions)` — "break this question into 3-5 focused sub-questions." 2. `src/rag/source_store.py`: `build_source_store()` — load `data/sources/*.md`, keep the filename as metadata, embed, add to Chroma. `src/rag/search.py`: `make_search(collection)` returns a `search(q) -> (text, source)` closure that retrieves the closest chunk and its `source` metadata.

Tests for this step: proven live in Step 3; for now, `make_search(collection)("your topic")` returns a `(text, source)` where `source` is one of your filenames.

Step 3 — The Pipeline (final)

What this step does: `run_research` ties it together — plan the question, build the search, execute the plan, compile the cited report — plus `main.py` and the integration test.

Why this step matters: this is one call that plans, researches every sub-question, and returns a cited report — the whole agent, assembled from three tested pieces.

What's new vs. Step 2: `src/pipeline.py`, `main.py`. **What stays the same:** everything else.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/pipeline.py`: `run_research(model, question)` — `make_plan` → `build_source_store` → `make_search` → `execute_plan` → `compile_report`. 2. Write `main.py` (take a question, print `format_report`) and `tests/integration/test_live_research.py`.

Final Deliverable: an agent that plans a broad question into sub-questions, researches each against your documents, and writes a report with citations.

Tests for this step: `test_live_research` — a broad question produces a report with several findings, each cited to a real source file.

Run, Test and Configure the Final Project

```
python main.py "Give me a briefing on retrieval-augmented generation"
python -m tests.unit.test_config
python -m tests.unit.test_report
python -m tests.integration.test_live_research
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity

Checklist Before You Call This Done

- [ ] The question is planned into sub-questions *before* any search runs
- [ ] Each finding is grounded in a retrieved document, with its source
- [ ] Every claim in the report is cited; sources are numbered and de-duplicated
- [ ] `execute_plan` takes a `search` function, so the engine tests with no model
- [ ] You can explain why "plan first, then execute each step" beats one big prompt

Publish It to GitHub

Name the repo `ResearchPilot-Planner-Executor-Research-Agent-With-Cited-Reports`, add the `README.md`, push. A citing research agent is a standout portfolio piece — it's what everyone wishes a chatbot did.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** before the solution. Build the executor + compiler (Step 1) first — the citation logic is the real exercise, and it needs no model.